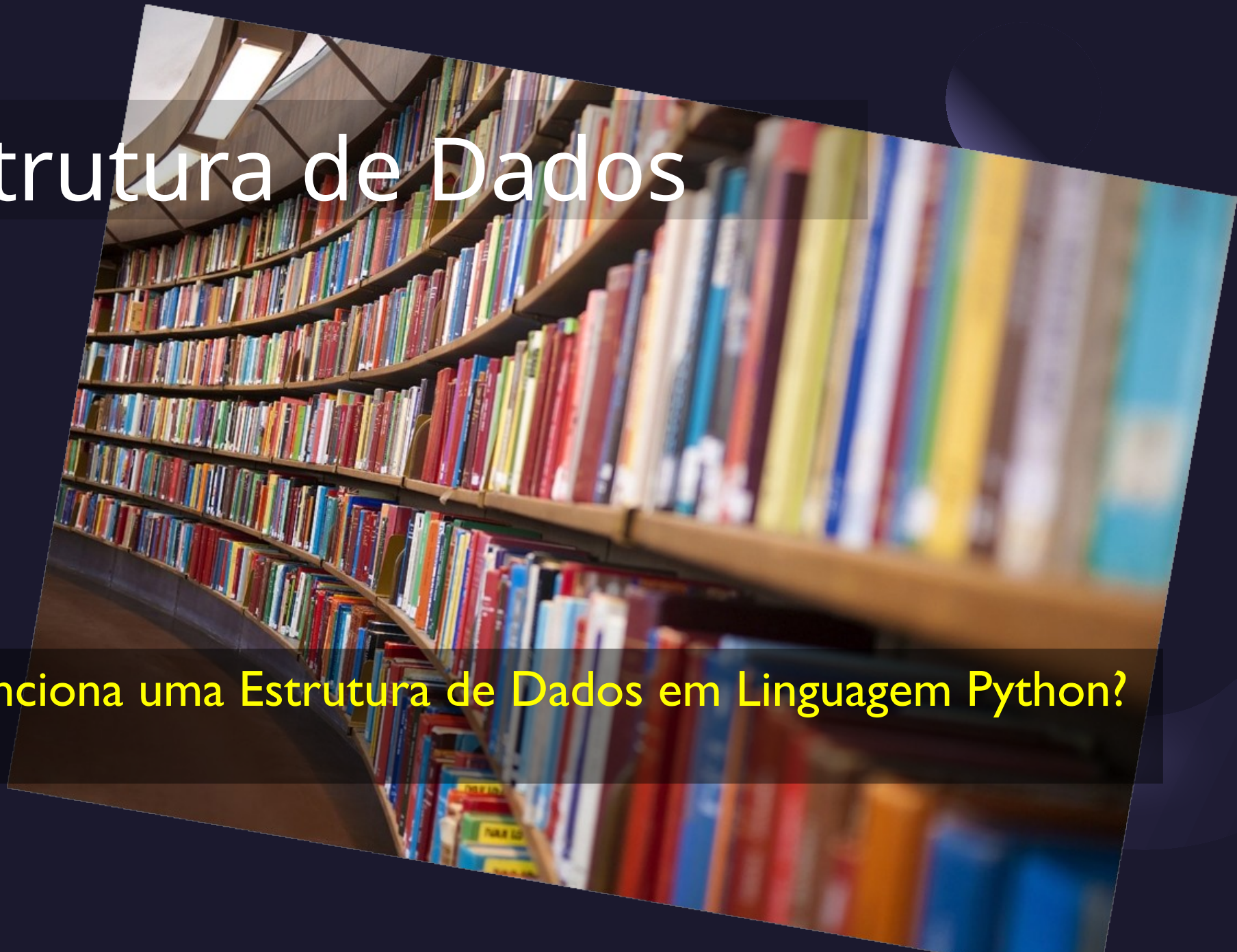
Linguagem de Programação Back-End

Prof. Wellington S. S. Silva

Python: Estrutura de Dados



Introdução: Como funciona uma Estrutura de Dados em Linguagem Python?

Estrutura de Dados– O que são e como funcionam

São formas de organizar e armazenar dados na memória para acesso e manipulação eficientes dessas informações.

Melhorando a performance do programa e dando mais robustez a aplicação que trará mais segurança e eficiência para o usuário final.

Coleção Fundamental de Estutura de Dados em Python

1. **Listas**
2. **Tuplas**
3. **Sets (Conjuntos)**
4. **Dicionário**

Estrutura de Dados - Listas

Uma **Lista** é uma estrutura de dados linear que armazena uma **sequência ordenada** de elementos do mesmo tipo. Pense em uma lista de compras, uma fila de banco ou os vagões de um trem: a característica principal é que existe uma ordem lógica entre os elementos.

Estrutura de Dados - Listas

Características Principais

Sequência Lógica: Os elementos estão organizados um após o outro. Mesmo que não estejam lado a lado na memória física do computador, eles têm uma relação de ordem (*predecessor* e *sucessor*).

Tamanho Variável (Geralmente): Diferentemente de um *Array* estático, uma lista pode **crescer ou diminuir** dinamicamente à medida que elementos são inseridos ou removidos (especialmente nas listas encadeadas).

Operações Comuns: As listas suportam operações básicas como:

- **Inserção:** Adicionar um novo elemento (no início, no fim ou em uma posição específica).
- **Remoção:** Excluir um elemento.
- **Busca (ou Acesso):** Encontrar ou acessar um elemento.

Estrutura de Dados - Listas

Tipos de Implementação de Listas

O modo como os dados são armazenados na memória define os principais tipos de listas:

1. Listas Sequenciais (ou Contíguas)

Como funciona: Os elementos são armazenados em **posições de memória fisicamente consecutivas** (em sequência).

Implementação: Geralmente são implementadas usando **Arrays (Vetores)**.

Vantagens:

- **Acesso Rápido (Acesso Direto/Constante):** Acessar um elemento por seu índice (x_i) é muito rápido, pois a posição de memória pode ser calculada diretamente.

Desvantagens:

- **Tamanho Fixo:** O tamanho máximo deve ser definido previamente (em algumas linguagens/contextos).
- **Movimentação de Dados:** Inserir ou remover um elemento no meio da lista é ineficiente, pois exige que todos os elementos seguintes sejam **movidos** para abrir espaço ou preencher a lacuna.

Estrutura de Dados - Listas

2. Listas Encadeadas (ou Ligadas)

Como funciona: Os elementos (**Nós**) podem estar espalhados pela memória. A ordem lógica é mantida através de **ponteiros** (ou referências). Cada **Nó** contém:

- O **Dado** (a informação).
- Um **Ponteiro** para o **próximo Nó** na sequência.

Vantagens:

- **Tamanho Dinâmico:** Crescem e diminuem facilmente, alocando memória apenas quando necessário.
- **Inserção/Remoção Eficiente:** Adicionar ou remover um Nó é rápido (basta reajustar os ponteiros), sem precisar mover os dados.

Desvantagens:

- **Acesso Sequencial:** Para acessar um elemento no meio da lista, é necessário **percorrer** a lista do início até ele (acesso lento).
- **Memória Extra:** Cada Nó precisa de espaço adicional para o ponteiro.

Estrutura de Dados - Listas

Variações de Listas Encadeadas

As listas encadeadas possuem variações importantes:

Tipo de Lista		Descrição
Simplesmente Encadeada	→	Cada nó aponta apenas para o **próximo** elemento. A navegação é unidirecional.
Duplamente Encadeada	→	Cada nó tem um ponteiro para o **próximo** e outro para o **anterior**. Permite navegação nos dois sentidos.
Circular	→	O ponteiro do último elemento aponta de volta para o **primeiro** elemento.

Estrutura de Dados - Listas

Relação com Outras Estruturas

Listas são tão flexíveis que são usadas para implementar outras Estruturas de Dados Abstratas (ADTs):

Pilha (Stack): Uma lista linear onde inserção e remoção ocorrem apenas em uma ponta, chamada **Topo**

(LIFO - *Last-In, First-Out*).

Fila (Queue): Uma lista linear onde a inserção ocorre em uma ponta (**Final**) e a remoção na outra (**Início**)

(FIFO - *First-In, First-Out*).

Em resumo, as listas são cruciais para organizar dados de forma sequencial, sendo a escolha da implementação (sequencial ou encadeada) ditada pelos requisitos de velocidade de acesso e eficiência nas operações de inserção/remoção.

Estrutura de Dados - Listas

Coleção ordenada e mutável

- Permite elementos duplicados

Sintaxe: lista = [1, 2, 3]

Principais operações:
- append(), insert(), remove(), pop(), sort(), len()

Exemplo:
frutas = ['maçã', 'banana', 'laranja']

Estrutura de Dados - Listas

Resumo das Funções:

Função	O que faz	Exemplo
len()	Retorna o número de elementos na lista.	len(minha_lista)
append()	Adiciona um item **ao final** da lista.	minha_lista.append('novo')
insert()	Adiciona um item em uma **posição (índice)** específica.	minha_lista.insert(1, 'meio')
remove()	Remove a **primeira ocorrência** de um **valor** específico.	minha_lista.remove('erro')
pop()	Remove e retorna um item pelo seu **índice**.	item = minha_lista.pop(0)
sort()	**Ordena** a lista (crescente ou alfabética) **no local**.	minha_lista.sort()

Estrutura de Dados - Listas

Exemplos - Praticar

Exercícios de Treinamento em Listas (Python)

Exercício 1: Controle de Estoque (Adição e Verificação)

Objetivo: Praticar append() e len().

Um pequeno armazém precisa controlar seu inventário.

1. Crie uma lista chamada estoque e inicialize-a com os seguintes produtos: "Camisa", "Calça", "Meia".
2. Use o método append() para adicionar três novos produtos: "Sapato", "Cinto", e "Gravata".
3. Imprima a lista estoque completa.
4. Use a função len() para calcular e imprimir quantos produtos estão agora no estoque.

Exercício 2: Gerenciamento de Tarefas (Inserção e Remoção)

Objetivo: Praticar insert(), remove() e pop().

Uma lista de tarefas (to-do list) precisa ser ajustada durante o dia.

1. Crie a lista tarefas inicial: "Comprar Pão", "Estudar Python", "Pagar Contas".
2. Um item urgente surgiu: use insert() para adicionar "Responder Email Urgente" como a **primeira** tarefa (índice 0).
3. Você terminou de estudar Python: use remove() para tirar "Estudar Python" da lista.
4. Um item no final foi descartado: use pop() (sem índice) para remover o último item da lista e imprima qual foi o item descartado.
5. Imprima a lista tarefas final.

Exercício 3: Pódio e Classificação (Ordenação e Acesso)

Objetivo: Praticar sort(), acesso por índice e len().

Você tem uma lista desordenada de pontuações de atletas.

1. Crie uma lista chamada pontuacoes com os seguintes números: [45, 88, 70, 92, 65, 88].
2. Use o método sort() para ordenar a lista **em ordem decrescente** (do maior para o menor). *Dica: use o argumento reverse=True.*
3. Imprima a lista pontuacoes após a ordenação.
4. Use o acesso por índice ([]) para imprimir a **maior pontuação** (o 1º colocado) e a **terceira maior pontuação** (o 3º colocado).

Estrutura de Dados - Tuplas

Uma **tupla** é uma **coleção ordenada** e **imutável** de elementos.

Ou seja:

Mantém a **ordem** em que os elementos foram inseridos.

Não pode ser alterada após sua criação (não é possível adicionar, remover ou modificar elementos diretamente).

Em outras palavras: é como uma lista, mas **congelada**.

Estrutura de Dados - Tuplas

Características Principais

Ordenada: Os elementos são mantidos em uma ordem definida e têm um índice.

Heterogênea: Pode armazenar diferentes tipos de dados em seus elementos (números, strings, outras listas ou tuplas, etc.).

Imutável (Immutable): Esta é a característica ***chave***. Após a criação, você **não pode** alterar, adicionar ou remover elementos de uma tupla.

Sintaxe: Geralmente é definida usando **parênteses ()** e os elementos são separados por vírgulas. Ex.: (1, 2, 3)

Estrutura de Dados – Tuplas

Use tuplas quando:

Você tiver um conjunto de dados que **não deve mudar** durante a execução do programa (por exemplo, as cores primárias, as coordenadas GPS de um local fixo, ou as configurações iniciais).

Você precisar de uma **chave em um dicionário** (dict): como as tuplas são imutáveis, elas podem ser usadas como chaves, enquanto listas não podem.

Você quiser garantir a **integridade dos dados** de forma simples, evitando modificações acidentais.

As tuplas são essenciais para representar dados fixos e estruturados, oferecendo uma forma mais segura e, em certos casos, mais eficiente de armazenar coleções de dados.

Estrutura de Dados – Tuplas vs Listas

É muito comum confundir Tuplas e Listas, pois ambas armazenam coleções ordenadas. A distinção mais importante é a **mutabilidade:**

Característica	Tupla (tuple)	Lista (list)
Sintaxe	Usam parênteses: (1, 2, 3)	Usam colchetes: [1, 2, 3]
Mutabilidade	**Imutável** (Não pode ser alterada)	**Mutável** (Pode ser alterada)
Uso Comum	Dados fixos, coordenadas (x, y), chaves de dicionário.	Coleções que mudam (estoque, tarefas, filas).
Performance	Geralmente mais **rápida** e ocupa menos memória.	Geralmente mais lenta em algumas operações.

Estrutura de Dados – Tuplas

Algumas da possibilidades de utilização para as **Tuplas**:

1. Criação e Acesso

Você acessa os elementos pelo índice, exatamente como em uma lista:

```
coordenada = (10, 25)

print(coordenada[0])                    # A Saída: 10
```

Você também pode criar uma tupla sem parênteses, apenas com vírgulas (O Python infere)

```
dados_pessoais = "Alice", 30, "Engenheira"

print(dados_pessoais)                   # A Saída será: ('Alice', 30, 'Engenheira')
```

Estrutura de Dados – Tuplas

Algumas da possibilidades de utilização para as **Tuplas**:

2. Desempacotamento (*Unpacking*)

Uma das utilizações mais elegantes das tuplas é o **desempacotamento**, onde você atribui os valores da tupla a múltiplas variáveis em uma única linha:

```
ponto = (3.5, 7.2, 1.0)

x, y, z = ponto              # O número de variáveis deve ser igual ao número de elementos da tupla


print(f"Valor de X: {x}")    # Saída: Valor de X: 3.5

print(f"Valor de Y: {y}")    # Saída: Valor de Y: 7.2
```

Estrutura de Dados – Tuplas

Algumas da possibilidades de utilização para as **Tuplas**:

3. Métodos Específicos

As tuplas possuem apenas dois métodos embutidos:

count(): Retorna o número de vezes que um valor aparece na tupla.

index(): Retorna o índice da primeira ocorrência de um valor.

Exemplo de Métodos Específicos em Tuplas

Vamos usar uma tupla chamada **notas_finais** que contém as notas de uma turma. Note que a nota 8.0 aparece mais de uma vez.

Estrutura de Dados – Tuplas

Exemplo de Métodos Específicos em Tuplas

```
# 1. Criação da Tupla (Imutável)

notas_finais = (10.0, 7.5, 8.0, 6.0, 9.5, 8.0, 5.0)

print(f"Tupla de Notas: {notas_finais}")

print("-" * 30)

# Queremos saber quantas vezes a nota 8.0 aparece.

contagem_8 = notas_finais.count(8.0)

print(f"Usando count(): A nota 8.0 aparece {contagem_8} vez(es).")

# Queremos saber onde a nota 9.5 está localizada.

indice_95 = notas_finais.index(9.5)

print(f"Usando index(): A nota 9.5 está no índice {indice_95}.")

#Vamos encontrar o índice da nota 8.0 (Ele retorna apenas o primeiro índice que encontra)

primeiro_indice_8 = notas_finais.index(8.0)

print(f"Usando index(): A primeira ocorrência de 8.0 está no índice {primeiro_indice_8}.")
```

Estrutura de Dados – Tuplas

Resumo das Saídas das **Tuplas:**

Método	**Resultado**	**Explicação**
count(8.0)	**2**	O valor 8.0 está presente duas vezes na tupla.
index(9.5)	**4**	O valor 9.5 é o 5º elemento, portanto, tem o índice 4.
index(8.0)	**2**	O valor 8.0 aparece primeiro no índice 2, e o método para de procurar ali.

Estrutura de Dados – Tuplas

Exercícios de Treinamento em Tuplas (Python)

Exercício 1: Dados Fixos e Acesso Rápido

Objetivo: Praticar a **criação** de tuplas e o **acesso por índice** (`[]`).

Crie uma tupla para armazenar os dados de um planeta que não devem mudar: nome, raio e densidade.

1. Crie uma tupla chamada `dados_planeta` com os seguintes valores: `"Marte"`, `3389.5` (raio em km) e `3.93` (densidade em g/cm³).

2. Imprima o **nome** do planeta usando o índice correto.

3. Imprima a **densidade** do planeta usando o índice correto.

4. Use o índice `-1` para imprimir o **último** elemento da tupla.

Estrutura de Dados – Tuplas

Exercícios de Treinamento em Tuplas (Python)

Exercício 2: Contagem e Localização de Elementos

Objetivo: Praticar os métodos de leitura `count()` e `index()`.

Em um registro de votação, alguns números de chapa apareceram repetidos.

1. Crie uma tupla chamada `votos` com os números: `(101, 205, 101, 300, 205, 101, 400)`.

2. Use o método `count()` para descobrir e imprimir quantas vezes a chapa `101` foi registrada.

3. Use o método `index()` para descobrir e imprimir a **primeira** posição (índice) onde o voto da chapa `300` aparece.

4. Tente usar o `append()` ou o acesso por índice para **adicionar** um novo voto (`500`) e observe o erro (`AttributeError` ou `TypeError`). *Seus alunos devem explicar por que isso acontece.*

Estrutura de Dados – Tuplas

Exercícios de Treinamento em Tuplas (Python)

Exercício 3: Desempacotamento de Dados

Objetivo: Praticar o **desempacotamento** (*unpacking*).

Uma função (que não precisamos criar) retorna as dimensões de uma caixa em uma única tupla.

1. Crie uma tupla chamada `dimensoes` com os valores: `(2.5, 5.0, 1.2)` (comprimento, largura, altura).

2. Use o **desempacotamento** para atribuir cada valor da tupla a três variáveis distintas, chamadas: `comprimento`, `largura` e `altura`.

3. Calcule e imprima a **área da base** da caixa, que é `comprimento * largura`.

4. Calcule e imprima o **volume** da caixa, que é `comprimento * largura * altura`.

Obrigado

Wellington S. S. Silva

guitomw@outlook.com